

Tides of Battle

Tides of Battle (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): *Tides of Battle* is a real-time, lane-based **1v1 strategy game** built around **faction deckbuilding**. Players pick a primary faction (defining playstyle + unique cards) and build a deck from that faction alone or **cross-faction** for new synergies, then bring it into a real-time arena to deploy units, manage resources and react to the opponent. Three core pillars: resource management, strategic deckbuilding, reactive decision-making. Team: 6 CS + 1 Design + 1 Art. Named technical features: **real-time multiplayer via deterministic lockstep** (both clients simulate an identical world locally, exchanging only tiny command packets); **fixed-timestep simulation decoupled from rendering**; **GPU-instanced sprite batching**; and a **data-driven content pipeline** (units/cards share attributes — health, speed, range, faction). Load-bearing C++: two simulation loops in parallel — a client-side loop and a server-side service — both running the exact same simulation with only player inputs transmitted; an **input-delay / frame-buffering** system ties inputs to a simulation frame, and both clients only advance a frame once both inputs are available (guaranteeing a synchronized state); the server manages connections, receives inputs and distributes them. Tool: a **network debugging tool** (player connection status, ping, current frame, input + buffer; plus controls to simulate packet loss / lag to test the input-delay system). Design features: ≥ 3 factions (distinct playstyle/strengths/identity); ≥ 8 units per faction; a mana-based resource system; and cross-faction deckbuilding. (*Monetization "can be integrated" via a gacha to unlock faction units.*)

Note (monetization — design condition): unlocking **gameplay units via gacha in a competitive 1v1** is pay-to-win, which breaks the module's clean-monetization rule. **If** monetization is added, keep all gameplay units earnable/free and restrict

real-money purchases to **cosmetics** (no pay-for-power in ranked play). Owned by the M2 Design goal D8 if pursued.

Note (team/scope): 1 artist for **3 factions × 8 units = 24+ unit sprites** (plus animations, UI and VFX) is a heavy load — scope the roster for the milestones (e.g. fewer units per faction at M1/M2, expand in T2), or consider adding a 2nd artist (the team has room to reach 9: 6 CS + 1 Design + 2 Art).

References:

- [Clash Royale — Wikipedia](#) (the benchmark **real-time 1v1 lane** card battler: deploy units from an 8-card deck, a steadily-regenerating **elixir/mana** resource, arenas + trophy ranking — the real-time-lane + mana + deck + PvP precedent)
- [Warcraft Rumble — Blizzard](#) (a real-time **multi-lane** collectible-unit battler with **faction leaders/families**, gold-based deployment and a rock-paper-scissors unit system — the real-time faction-army lane precedent)
- [Stormbound on Steam](#) (a lane-based **faction deckbuilding** card battler — four kingdoms with distinct synergies, mana, and competitive PvP ladder — the faction-deckbuilding + cross-synergy precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Tides of Battle
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	✓

How each requirement applies:

Platform fit: **Main:** Native mobile (device: Pixel 9a) and/or Web browser on mobile — a real-time 1v1 card-strategy game suits either; the Native PC build is the development/tools vehicle (the network debug tool + the content editor).
Extended: Firebase (accounts, collection, matchmaking/rating) + a Linux server the team sets up by M2 (full use T2) as the ****lockstep relay**** (manages connections, receives + distributes input packets).

Front end: A 2D renderer with **GPU-instanced sprite batching** for the many units/projectiles across the lanes, the deck/hand + mana HUD, the deckbuilder and collection screens — decoupled from the fixed-timestep sim.

Languages: Native — Kotlin Native Android UI + C++ via JNI (Android Studio); Web — HTML/JS/CSS + C++→WASM (Emscripten); OpenGL ES sprite instancing in a GLSurfaceView where realtime 2D is needed (no SDL on native).

Back end: The **deterministic fixed-timestep C++ simulation** (units, lanes, mana, combat) running identically on both clients, driven by lockstep input packets with frame-buffering; a **server-side loop/service** managing connections and relaying inputs; plus accounts/collection/rating. **Languages:** C++ (client sim + server relay); Firebase-backed services on the Extended tier.

Backend scope: Server-indicated (T2) — the lockstep relay + accounts/collection seam set up & basic-tested on the Linux host by M2, full use in T2.

- *Loop and/or service:* the C++ loop is heavily load-bearing — a **deterministic fixed-timestep** simulation of units/lanes/mana/combat, decoupled from rendering, that advances a frame only once both players' inputs for that frame are buffered; the server-side service relays inputs. Determinism is the make-or-break property.
- *Data-driven:* units/cards (health, speed, range, faction, abilities), the ≥ 3 factions and cross-faction rules, and mana costs are all data via the content pipeline — a new unit/faction is a data change.
- *2D:* GPU-instanced sprite rendering of the lane battlefield, units, projectiles and the deck/mana HUD (custom-rendered). *3D:* not needed. *Networking:* ✓ **the signature** — real-time deterministic lockstep with input-delay/frame-buffering and a server relay (one of the few projects where a full real-time Networking System is central).
- *Custom tool:* the **network debugging tool** (connection/ping/frame/input-buffer inspection + simulated packet-loss/lag) is excellent dev infrastructure; the team should also build a **content-authoring editor** (unit/card/faction editor on the data pipeline) as the graded content tool (see M2G2 ★).
- *External DB:* accounts, the card collection, deck configs and matchmaking/rating. *Auth »:* accounts for ranked PvP. *Analytics »:* optional — faction/card win-rates for balancing.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the **deterministic fixed-timestep** sim decoupled from rendering.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a deterministic fixed-timestep C++ + simulation of units/lanes/mana/combat, decoupled from the render loop, with an input-frame model ready for lockstep; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** M1G11 (lockstep consumes the deterministic sim); every other goal.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System** — the data-driven content pipeline (units/cards/factions).

Tech & dependencies

- **Tech:** Entity/component model with units/cards sharing attributes (health, speed, range, faction, abilities), the ≥ 3 factions + cross-faction rules and mana costs in hot-reloadable JSON — a new unit/faction is a data change.
- **Prerequisites:** G1 (loop & sim).
- **Feeds into:** M2G2 (the content editor), M2G13 (mana/deck systems).

- **G3 ★ Debugging, Profiling, Stress Test & CI** — includes the **network debugging tool** + a desync detector.

Tech & dependencies

- **Tech:** ImGui overlays + the network debug tool (connection/ping/frame/input-buffer, simulate packet-loss/lag), a per-frame state-hash desync detector, Tracy profiling of a full-lane fight, Catch2 tests proving

deterministic replay (same inputs → identical state), on a GitHub Actions matrix.

- **Prerequisites:** G1 (sim to instrument).
- **Feeds into:** M2G3 (CI hardening); the network debug tool is a named deliverable; determinism is verified here.

- **G4 2D Graphics** — the lane renderer with **GPU-instanced sprite batching**.

Tech & dependencies

- **Tech: Web build:** A 2D renderer using GPU-instanced sprite batching for the many lane units/projectiles, a camera, and the deck/hand/mana HUD, decoupled from the sim; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (sim to visualise).
- **Feeds into:** M2G4 (advanced VFX); every visible frame.

- **G8 Enemy AI & Pathfinding** — the **lane unit behaviour** (units auto-advance, target and attack down their lane).

Tech & dependencies

- **Tech:** Deterministic per-unit behaviour (advance lane, acquire target, attack, ability triggers) + lane pathing, all driven by the unit's data (range/speed/faction), computed inside the fixed-timestep sim (custom C++).
- **Prerequisites:** G1 (tick), G2 (unit data).
- **Feeds into:** The lane combat; M2G13 (unit/faction systems). (This is unit behaviour, not a PvE opponent — the opponent is the human via lockstep.)

- **G11 Networking Foundation** — the **deterministic lockstep** transport + input-delay/frame-buffering + server relay.

Tech & dependencies

- **Tech: Web build:** Sockets (Winsock2/BSD, or ENet) exchanging tiny per-frame input command packets, an input-delay/frame-buffer that advances the sim only when both inputs are present, and a server relay (connections, receive, distribute); sequence/lockstep confirmation. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView

where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G1 (deterministic sim), G3 (desync detection).
- **Feeds into:** M2G7 (lockstep hardening under latency); the whole real-time-multiplayer pillar.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling of a full-lane fight, ASan on MSVC, pools for units/projectiles, GPU-instancing tuning, and on-device validation (Pixel 9a and/or mobile web); the Linux lockstep-relay server + Firebase (accounts/collection) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The mobile deliverable + a stable lockstep match on-device.

- **G2 ★ Content Editor — Core (Unit / Card / Faction Editor) —** author units/cards (health/speed/range/abilities), factions and cross-faction rules on the data pipeline.

Tech & dependencies

- **Tech:** Dear ImGui editor to define units/cards, assign them to factions, set mana costs and abilities, and preview them on the lane, with round-trip JSON and undo/redo. (The network debug tool from G3 ★ is the second, dev-facing tool.)
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (preview).
- **Prerequisites:** G2 (the data model), M1G4 (rendering for preview).
- **Feeds into:** All unit/faction content; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible mobile builds, tests (deterministic replay, lockstep sync under simulated latency/loss, data loading) and packaging via GitHub

Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G7 Networking — Match Sync & Prediction** — the lockstep hardened: input-delay tuning, stall/recovery, desync handling under real latency/loss. Tech & dependencies

- **Tech:** Lockstep input-delay tuning, handling of dropped/late packets (stall + catch-up), per-frame state-hash desync detection + recovery, tested under the network debug tool's simulated packet-loss/jitter/lag.
- **Prerequisites:** M1G11 (lockstep foundation), M2G1 (optimization).
- **Feeds into:** A robust real-time 1v1 match at showcase.

- **G13 Gameplay Systems II** — the **mana economy + faction / cross-faction deckbuilding + unit deployment** systems.

Tech & dependencies

- **Tech:** C++ systems over the ECS for the mana resource, unit deployment, faction identities (strengths/weaknesses/mechanics) and cross-faction synergy rules; balancing-data-driven.
- **Prerequisites:** M1G2 (unit/faction data), M1G8 (unit behaviour).
- **Feeds into:** The deckbuilding + reactive-combat depth at showcase; T2 faction/unit expansion.

- **G8 Firebase — Accounts, Leaderboards & Analytics** — accounts, the card collection, deck configs and the ranked matchmaking/leaderboard.

Tech & dependencies

- **Tech:** Firebase Auth accounts, Firestore for the collection + deck configs + matchmaking/rating leaderboard, and optional Analytics, via the REST API from C++; server-validated results.
- **Prerequisites:** G2 (collection/deck data model).
- **Feeds into:** Ranked 1v1 + persistent collection; balancing telemetry.

Track Goals & Team Composition

Recommended composition: the team's stated **6 CS + 1 Design + 1 Art** (8) is workable, but flag the **art load**: 3 factions $\times$ 8 units = 24+ unit sprites plus animations, UI and VFX is a lot for one artist. One designer owns the faction identities, the mana economy, the cross-faction deckbuilding and the competitive balance; the artist carries the faction unit rosters, the lane battlefield and the deck/collection UI. **Recommendation:** either scope the unit roster down for M1/M2 (fewer units per faction, expand in T2) or add a **2nd artist** to reach 6 CS + 1 Design + 2 Art (9) (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). *Scope note (advisory):* deterministic lockstep + a deterministic sim + GPU-instancing + factions/deckbuilding + a content editor + matchmaking is a large surface — a deterministic 1-lane 1v1 with ~2 factions and a handful of units, the lockstep netcode and the content editor is the M1/M2 floor; all 3+ factions $\times$ 8 units, ranked matchmaking and the gacha (if any) are the stretch. **Determinism across both clients is the single highest technical risk — prove it early (state-hash desync detection in M1).**

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the real-time lane 1v1, the faction identities, the mana economy and cross-faction deckbuilding; primary benchmark e.g. Clash Royale / Stormbound, with a Singapore benchmark for the local-awareness criterion)
- M1D2 Core Gameplay Loop Prototype (a greybox proving deploy $\rightarrow$ lane combat $\rightarrow$ react with mana is fun in real time) · prototype in Unity
- M1D6 Systems & Economy Design (the ≥ 3 faction identities + ≥ 8 units each, the mana economy, and the cross-faction synergy rules — every system with a purpose; balance-first given 1v1 PvP)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (faction/unit/mana balance and matchmaking tuning via the team's JSON, no code changes) — *(if monetization is pursued, pair with D8 Monetization & Scope Integrity to keep it non-pay-to-win)*

- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the lane battlefield, the hand/mana HUD and the deckbuilder/collection screens.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — the arena, deckbuilder, collection and matchmaking screens navigable unaided on the mobile target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished deploy/combat + deckbuilding UX tested with players on the Pixel 9a.
- **Art M1:** 1 artist (scope the roster — or add a 2nd artist; see composition)
 - M1A1 ★ Art Bible / Proof of Concept (the visual direction + the per-faction identity language — silhouettes, palettes, card frames)
 - M1A2 Character Concepts & Model Sheets (the faction unit rosters — prioritise a readable, reusable per-faction kit)
 - M1A4 UI Art Concepts & Typography (lane battlefield, hand/mana HUD, deckbuilder/collection screens)
- **Art M2:**
 - M2A1 ★ Critical Asset Production & In-Engine Visual Quality (units, battlefield and UI implemented and consistent)
 - M2A2 Character Animation — Extensive (unit move/attack/ability/death across the factions)
 - M2A5 VFX — Critical Effects & Transitions (deploy, ability, impact and destroy effects)
- **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (faction/battle BGM, deploy, attack, ability and mana cues), suitable and correctly triggered.
 - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for a busy lane.

• **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (faction BGM + deploy / attack / ability / mana / victory SFX, ≥6 types), scaling cleanly with lane activity.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; distinct per-faction/ability cues).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the content editor + the network debug tool), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*